

# Week 2 Mini Project: Student Goal Tracker

# Introduction

By this point in the course, students have learned about:

- variables
- input and output
- loops
- functions

This mini project gives them a chance to combine those skills into one small but meaningful Python program.

In this project, students will build a simple **Student Goal Tracker**. The program will ask the user to enter several learning goals, store them, and then display them back in a clean format.

This is a great beginner project because it introduces a pattern used in real software:

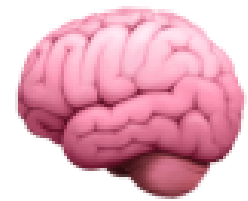
- get input
- process data
- display results

It also helps students practice using **functions to organize code** and **loops to repeat actions**.

# **Project Objective**

Create a Python program that:

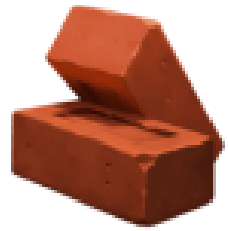
- asks the user how many learning goals they want to enter
- uses a loop to collect those goals
- uses functions to organize the program
- displays the goals back to the user in a numbered list



# Skills Practiced

This mini project reinforces:

- defining functions
- calling functions
- using parameters
- using return values
- using a `for` loop
- storing values in a list
- printing formatted output



# Program Requirements

Your program must:

1. Ask the user for their name
2. Ask how many learning goals they want to enter
3. Use a loop to collect that many goals
4. Store the goals in a list
5. Use functions to organize the code
6. Display a summary showing the student's name and goals



# Sample Output



```
Enter your name: Eric
How many learning goals do you want to enter? 3
Enter goal 1: Learn Python basics
Enter goal 2: Understand loops
Enter goal 3: Build a small app
Hello, Eric
Here are your learning goals:
1. Learn Python basics
2. Understand loops
3. Build a small app
```



# Suggested Program Design

Encourage students to break the program into smaller pieces using functions.  
For example:

- one function to get the student's name
- one function to get the number of goals
- one function to collect the goals
- one function to display the results

This helps students see how functions make programs easier to build and understand.



# Complete Example Solution

```
def get_student_name():
    name = input(Enter your name: )
    return name
def get_goal_count():
    count = int(input(How many learning
goals do you want to enter? ))
    return count
def collect_goals(count):
    goals = []
    for i in range(count):
        goal = input(fEnter goal {i + 1}:
    )
        goals.append(goal)
    return goals
def display_goals(name, goals):
    print()
    print>Hello,, name)
    print(Here are your learning goals:)
    print()
    for i in range(len(goals)):
        print(f{i + 1}. {goals[i]})
def main():
    student_name = get_student_name()
    goal_count = get_goal_count()
    goal_list = collect_goals(goal_count)
    display_goals(student_name,
goal_list)
main()
```



# Code Explanation

## 1. `get_student_name()`



```
def get_student_name():  
    name = input(Enter your name: )  
    return name
```

This function asks the user for their name and returns it to the main program.

## 2. `get_goal_count()`



```
def get_goal_count():  
    count = int(input('How many learning goals do you want to enter? '))  
    return count
```

This function asks the user for the number of goals they want to enter.

Because `input()` returns text, we use `int()` to convert the value into a number.

### 3. `collect_goals(count)`



```
def collect_goals(count):  
    goals = []  
    for i in range(count):  
        goal = input(f'Enter goal {i + 1}: )  
        goals.append(goal)  
    return goals
```

This function uses a `for` loop to repeat the input step multiple times.

## Important ideas here:

- `goals = []` creates an empty list
- `range(count)` repeats the loop the correct number of times
- `append()` adds each new goal to the list

#### 4. `display_goals(name, goals)`



```
def display_goals(name, goals):  
    print()  
    print(Hello,, name)  
    print(Here are your learning goals:)  
    print()  
    for i in range(len(goals)):  
        print(f{i + 1}. {goals[i]})
```

This function displays the final results.

It uses a loop to go through the list and prints each goal with a number.

## 5. `main()`



```
def main():  
    student_name = get_student_name()  
    goal_count = get_goal_count()  
    goal_list = collect_goals(goal_count)  
    display_goals(student_name, goal_list)
```

The `main()` function controls the overall flow of the program.

This is a great habit for students because it keeps the program organized.



# How to Run the Program

Save the file as:



```
student_goal_tracker.py
```

Then run it from the terminal.

## On Mac or Linux

```
python3 student_goal_tracker.py
```

## On Windows

```
python student_goal_tracker.py
```



# Why This Is a Good Week 2 Project

This project is a strong fit for Week 2 because it is simple enough for beginners, but it still introduces real programming structure.

Students learn that:

- loops are useful when repeating input
- functions make code easier to manage
- lists can store multiple values
- a program can be built one small piece at a time

This helps move them from writing isolated examples to building an actual program.



# Suggested Enhancements

Once students complete the main version, they can add more features.

## Enhancement 1 — Add a Motivational Message

After displaying the goals, print a message like:

```
print()  
print(Keep working toward your goals!)
```

## Enhancement 2 — Ask for a Course Name

Students can ask:

```
course = input(Enter the name of your course: )
```

Then include it in the summary.

## Enhancement 3 — Display Total Number of Goals

Students can print:

```
print(You entered, len(goals), goals.)
```

These small improvements reinforce the idea of iterative development.

# **Assignment Instructions**

## **Week 2 Mini Project — Student Goal Tracker**

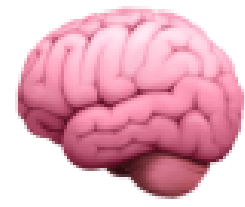
Write a Python program that does the following:

1. asks the user for their name
2. asks how many learning goals they want to enter
3. uses a loop to collect each goal
4. stores the goals in a list
5. uses functions to organize the code
6. displays the completed list of goals back to the user

## **Submission Requirements**

Students should submit:

- their Python source code
- a copy of the output from running the program



# Summary

In this mini project, students combine **loops** and **functions** to build a small but useful Python application.

They practice:

- using functions to break a problem into smaller parts
- using loops to repeat actions
- collecting and storing user input
- displaying results in a structured way

This is an important step because it shows students how separate Python concepts work together in a real program.



# Final Thought

This kind of project is where students start feeling like real programmers.

They are no longer just typing isolated examples. They are building a complete program with structure, flow, and user interaction.

That is a major milestone in Week 2.

I can also turn this into a full **lab handout with objectives, step-by-step instructions, grading rubric, and challenge section.**